

Relay — Architecture Decision Records (ADRs)

Short records of *why* we chose what we chose. Format: Context → Decision → Consequences.
Newest first.

ADR-005 — Claude as the primary synthesis model

Status: Accepted **Context:** Cards must be short, grounded strictly in retrieved chunks, and never hallucinate. We need reliable instruction-following on "answer only from these sources, else return no card." **Decision:** Use Claude for card synthesis; keep Minimax as an optional secondary (track tie-in + fallback). **Consequences:** Strong grounding adherence; one less variable in the demo. Cost is fine at hackathon volume. Model is swappable behind the orchestrator interface.

ADR-004 — Cards on a separate dashboard, not an overlay

Status: Accepted **Context:** Live-assist tools that overlay/inject into the call raise consent and "undetectable" concerns, and complicate the demo. We want a clean, defensible presentation. **Decision:** Render cards on a companion dashboard beside the call rather than overlaying the other party's screen. Whisper-back (TTS) is an optional stretch, not the default. **Consequences:** Ethically cleaner story for judges; simpler to build and stage; clear split-screen demo. Slightly less "magical" than an in-ear whisper, but more credible as a product.

ADR-003 — One engine, three modes (not three products)

Status: Accepted **Context:** Prizes are awarded across multiple tracks (Co-Pilot, Support, Lead Gen). Building three products in 24h is impossible; building one is feasible. **Decision:** Build a single ambient retrieval engine and expose three thin modes (Live / Desk / Intake) that differ only in trigger logic and presentation. **Consequences:** Maximum prize surface area for minimum extra build. Risk: scope creep across modes — mitigated by SCOPE.md cut-lines (Live is the protected core).

ADR-002 — Python + FastAPI + LiveKit Agents for the backend

Status: Accepted **Context:** The core is a real-time audio → retrieval → synthesis loop. We need first-class live-audio tooling and an async-friendly server. **Decision:** Python 3.11 with FastAPI (REST + WebSocket) and the LiveKit Agents framework for the worker. React + TypeScript on the frontend. **Consequences:** Best-supported path for LiveKit + the ML/LLM ecosystem; async fits the streaming workload. Team works in two well-separated languages with a frozen API contract between them.

ADR-001 — Moss for retrieval (over pgvector/Pinecone as primary)

Status: Accepted **Context:** A natural voice reply budget is ~200 ms; cold vector lookups run 97–307 ms, which alone blows the budget. Real-time grounding is impossible if retrieval is slow. Moss targets <10 ms. **Decision:** Use Moss as the primary retrieval index. Keep pgvector in Postgres as a demo-safe fallback. **Consequences:** Retrieval stops being the bottleneck — this is the technical moat and the reason the demo feels instant. Also aligns with the host sponsor (judge goodwill). Dependency on a sponsor service is mitigated by the pgvector fallback.

Template for new ADRs

```
## ADR-00X — <title>
**Status:** Proposed | Accepted | Superseded by ADR-00Y
**Context:** <forces at play>
**Decision:** <what we chose>
**Consequences:** <tradeoffs, follow-ups>
```